

Diskretne strukture

Iztok Peterin

RIT, IPT, FERI, UM

5. poglavje: Časovna zahtevnost

Definicija algoritma

- ▶ Problemi, ki jih rešujem z računalniki, so vedno zahtevnejši.
- ▶ Zato potrebujemo čim hitrejša algoritme za njihovo reševanje.
- ▶ Kako ugotoviti kateri algoritem je boljši?
- ▶ Kaj algoritem sploh je?
- ▶ Temeljna značilnost vseh metod, ki se izvajajo na računalniku je, da mora biti postopek **končen**.
- ▶ Algoritem na koncu svojega izvajanja poišče **rešitev problema**, ki ga algoritem rešuje.
- ▶ Algoritem ne more določiti rešitve problema iz nič.
- ▶ Tako potrebujemo **vhodne podatke**, ki jih lahko vnesemo preko tipkovnice, ali pridobimo iz elektronskega vira.
- ▶ **Algoritem za problem P** je postopek, ki v končno mnogo korakih iz vhodnih podatkov določi rešitev problema P , ki je podana na izhodu.

Velikost vhodnih podatkov

- ▶ S čim merimo kvaliteto algoritma?
- ▶ Algoritem za enak problem ni enako hiter, če mu na vходу ponudimo majhen oziroma velik vzorec.
- ▶ Tako za pregled instagram povezav predsednika države, potrebujemo več časa, kot če preverjamo člane lokalnega gobarskega društva.
- ▶ Merilo naj bo odvisno od **velikosti vhodnih podatkov**, saj potem algoritem ocenjujemo neodvisno od njih.
- ▶ Velikost vhodnih podatkov označimo z VVP.
- ▶ VVP običajno predstavimo z velikostnim redom, ki ga vhodni podatki zavzemajo pri hranjenju v računalniškem spominu.

Deranžacije 1

- ▶ Kako določiti merilo za kvaliteto algoritma?
- ▶ V definiciji za algoritem se skriva fraza 'v končno mnogo korakov'.
- ▶ Tako preštejemo število korakov, ki jih algoritem opravi.
- ▶ To število korakov nato izrazimo s funkcijo f , ki je odvisna od velikosti vhodnih podatkov.
- ▶ Ker nam funkcija f šteje korake, njeno definicijsko območje predstavljajo naravna števila in imamo $f : \mathbb{N} \rightarrow \mathbb{R}$.
- ▶ Ob tem nas ne zanima $f(n)$, pač pa $f(VVP)$.

Shranjevanje naravnega števila 1

- ▶ Kako je v računalniku skranjeno število, ki ima v desetiškem številskem sistemu obliko $n = (n_k n_{k-1} \dots n_1 n_0)_{10}$.
- ▶ Vsa števila so v računalniku shranjena v dvojiški obliki, zato nas zanima $n = (b_\ell b_{\ell-1} \dots b_1 b_0)_2$, kjer je $b_i \in \{0, 1\}$ za vsak $i \in [\ell]_0$.
- ▶ Zanima nas število ℓ , saj predstavlja število bitov, ki jih potrebujemo, da shranimo n .

Algorithm 1: Pretvorba števila iz desetiškega v binarni zapis

Data: Število z zapisano v desetiškem sistemu.

Result: Število $b = z$ zapisano v binarnem sistemu.

$b = 0, i = 0$

while $z > 0$ **do**

$b_i = z \pmod{2}$
 $z = z \operatorname{div} 2$
 $i = i + 1$

end

- ▶ Ukaz $(\operatorname{mod} 2)$ pomeni ostanek pri deljenju števila z 2 in je lahko le 0, ko je naše število sodo, in 1, ko je naše število liho.
- ▶ Ukaz $(\operatorname{div} 2)$ dano število deli z 2 in kot rezultat ohrani le celo število brez decimalnega dela (če ta sploh obstaja).

Shranjevanje naravnega števila 2

Algorithm 2: Pretvorba števila iz desetiškega v binarni zapis

Data: Število z zapisano v desetiškem sistemu.

Result: Število $b = z$ zapisano v binarnem sistemu.

$b = 0, i = 0$

while $z > 0$ **do**

$b_i = z \pmod{2}$

$z = z \text{ (div } 2)$

$i = i + 1$

end

- ▶ Zanima nas, kolikokrat se izvrši zanka iz algoritma.
- ▶ Vsakič, ko se zanka izvrši, smo število n razpolovili.
- ▶ Oziroma, da je n večji od ustrezne potence 2.
- ▶ Če se je zanka izvršila k -krat, to pomeni, da je $2^{k-1} \leq n < 2^k$, oziroma $k - 1 \leq \lg n < k$, kjer $\lg$ označuje dvojiški logaritem.
- ▶ Tako je število bitov, ki jih potrebujemo v računalniku, da shranimo naravno število n , navzdol omejeno z $\lg n$.
- ▶ Ob tem je $\lg n = k - 1$ natanko tedaj, ko je $n = 2^{k-1}$ in takrat je $k = 1 + \lg n$.

Asimptotične meje

- ▶ Naj bosta f in g funkciji, ki obe slikata iz naravnih števil v realna.
- ▶ Oglejmo si, v kakšnem razmerju sta lahko funkciji f in g , ko gre za dovolj velika naravna števila.
- ▶ Rečemo, da je $f(n)$ **reda največ** $g(n)$, ali $f(n)$ **je veliki O od $g(n)$** , kar pišemo $f(n) = O(g(n))$, če obstaja konstanta $C_1 \in \mathbb{R}$, da je $f(n) \leq C_1 g(n)$ za vsa naravna števila, razen končno mnogo.
- ▶ Če je $f(n) = O(g(n))$, potem je $g(n)$ **asimptotična zgornja meja** funkcije $f(n)$.
- ▶ Rečemo, da je $f(n)$ **reda vsaj** $g(n)$, ali $f(n)$ **je omega Ω od $g(n)$** , kar pišemo $f(n) = \Omega(g(n))$, če obstaja konstanta $C_2 \in \mathbb{R}$, da je $f(n) \geq C_2 g(n)$ za vsa naravna števila, razen končno mnogo.
- ▶ Če je $f(n) = \Omega(g(n))$, potem je $g(n)$ **asimptotična spodnja meja** funkcije $f(n)$.
- ▶ Rečemo, da je $f(n)$ **reda** $g(n)$, ali $f(n)$ **je theta Θ od $g(n)$** , kar pišemo $f(n) = \Theta(g(n))$, če je $f(n)$ hkrati reda največ in reda vsaj $g(n)$.
- ▶ Torej, če je hkrati $f(n) = O(g(n))$ in $f(n) = \Omega(g(n))$.
- ▶ Če je $f(n) = \Theta(g(n))$, potem je $g(n)$ **natančna asimptotična meja** funkcije $f(n)$.

Dva izreka

Izrek

Za polinom $p_k(n) = a_k n^k + a_{k-1} n^{k-1} + \dots + a_1 n + a_0$ velja $p_k(n) = \Theta(n^k)$.

Dokaz.

Izrek

Če je $f(n) = O(g(n))$, potem je $f(n) + g(n) = \Theta(g(n))$.

Dokaz.

Časovna zahtevnost algoritma

- ▶ Algoritem za problem P ima **časovno zahtevnost** $f(n) = O(g(VVP))$, če ga izvedemo v $f(n)$ korakih glede na velikost vhodnih podatkov VVP .
- ▶ Torej je za določitev časovne zahtevnosti algoritma potrebno prešteti število operacij v algoritmu.
- ▶ S tem velja, da ima tudi problem P **časovno zahtevnost** $O(g(VVP))$, saj imamo algoritem v takšni časovni zahtevnosti, ki ta problem reši.
- ▶ Do spodnje asimptotične meje $\Omega(g_1(n))$ problema P se ne moremo prebiti s pomočjo algoritma, pač pa poiščemo lastnost, zaradi katere problema P ni moč rešiti hitreje.
- ▶ To je pogosto precej težje, kot najti algoritem, ki ni nujno optimalen.
- ▶ Kaj storimo, če algoritem ni optimalen?
- ▶ Poskusimo najti boljšega.
- ▶ Če ga najdemo, potem ima nov algoritem boljšo časovno zahtevnost in s tem se izboljša tudi časovna zahtevnost samega problema.

Vrste časovnih zahtevnosti

- ▶ Omenimo najpomembnejše časovne zahtevnosti, ki so zbrane v naslednjem seznamu. Ob tem velikost vhodnih podatkov označimo z n .

$O(1)$	konstantna časovna zahtevnost,
$O(\lg n)$	logaritemska časovna zahtevnost,
$O(\sqrt{n})$	korenska časovna zahtevnost,
$O(n)$	linearna časovna zahtevnost,
$O(n \lg n)$	časovna zahtevnost $n \lg n$,
$O(n^2)$	kvadratna časovna zahtevnost,
$O(n^3)$	kubična časovna zahtevnost,
$O(n^k)$	polinomska časovna zahtevnost,
$O(2^n)$	eksponentna časovna zahtevnost,
$O(n!)$	fakultetna časovna zahtevnost.

Dejanski čas izvajanja

- ▶ Za hitre oziroma obvladljivi algoritmi imajo polinomsko ali boljšo časovno zahtevnostjo in še tukaj težimo k čim manjši potenci.
- ▶ Nikakor pa ne želimo algoritmov z eksponentno ali višjo časovno zahtevnostjo.
- ▶ V spodnji tabeli je predstavljenih nekaj časovnih zahtevnosti algoritmov skupaj s konstantami in čas, ki ga ti algoritmi potrebujejo za izračun na ustrezno velikem vzorcu.
- ▶ Ob tem predpostavimo, da se ena operacija v povprečju izvrši v eni nano sekundi, to je $1ns = 10^{-9}s$.
- ▶ Povedano drugače, v eni sekundi se izvrši milijarda operacij.

n	$100n \lg n$	$10n^2$	$n^{3.5}$	$n^{\lg n}$	2^n	$n!$
10	$3\mu s$	$1\mu s$	$3\mu s$	$2\mu s$	$1\mu s$	$4ms$
20	$9\mu s$	$4\mu s$	$36\mu s$	$420\mu s$	$1ms$	76 let
30	$15\mu s$	$9\mu s$	$148\mu s$	$20ms$	$1s$	$8 \cdot 10^{15}$ l
50	$28\mu s$	$25\mu s$	$884\mu s$	$4s$	13 dni	
100	$66\mu s$	$100\mu s$	$10ms$	$5h$	$4 \cdot 10^{13}$ let	
1000	$1ms$	$10ms$	$32s$	$3 \cdot 10^{13}$ let		
10^6	$2s$	$3h$	3169 let			
10^{10}	$9h$	$3 \cdot 10^4$ let				

Zgledi

- ▶ Koliko prostora potrebujemo, da v računalniku shranimo s naravnih števil $a_1, a_2, \dots, a_s$?
- ▶ Za algoritem Bubble sort smo ugotovili, da velja $b_n = \frac{1}{2}(n^2 - n)$. Zato je časovna zahtevnost algoritma Bubble sort kar

$$O(n^2) = \dots .$$

- ▶ Časovna zahtevnost pretvorbe decimalnega zapisa naravnega števila n v dvojiški zapis je $VVP = O(\lg n)$.

Računanje potence

- ▶ Zapišimo algoritem, ki izračuna potenco a^k za podana $a \in \mathbb{R}$ in $k \in \mathbb{N}$.

Algorithm 3: Računanje potence

Data: Naravno število k in realno število a .

Result: $ans = a^k$.

$ans = a$

for $i = 2$ **to** k **do**

$ans = a \cdot ans$

end

Hitro računanje potence

- ▶ Ta algoritem izboljšamo ob upoštevanju dvojiškega zapisa števila $k = (b_n b_{n-1} \dots b_1 b_0)_2$.

Algorithm 4: Hitro računanje potence

Data: Naravno število v dvojiškem zapisu $k = b_n b_{n-1} \dots b_1 b_0$ in realno število a .

Result: $ans = a^k$.

$ans = 1$ in $temp = a$

for $i = 0$ **to** n **do**

if $b_i = 1$ **then**

$ans = ans \cdot temp$

end

$temp = temp \cdot temp$

end

Ali je n praštevilo

- ▶ Zapišimo preprost algoritem, ki preveri, ali je naravno število n praštevilo.

Algorithm 5: Praštevilo

Data: Naravno število n .

Result: $b = 1$, če je n praštevilo in $b = 0$ sicer.

$b = 1$

```
for  $i = 2$  to  $n - 1$  do  
    if  $n = 0 \pmod i$  then  
         $b = 0$   
         $i = n - 1$   
    end  
end
```

- ▶ Zgornji algoritem lahko izboljšamo, če uporabimo enostavno dejstvo, da če k deli n , potem tudi $\frac{n}{k}$ deli n .
- ▶ Ob tem je eden zagotovo manjši ali enak $\sqrt{n}$.
- ▶ Tako zadošča, da zgornjo mejo v for zanki zamenjamo s $\lfloor \sqrt{n} \rfloor$.